

03-ad-hoc 和常用模块

1 ad-hoc 介绍

ansible 执行方式的其中一种是 ad-hoc，ad-hoc 就像一条 Linux 命令，ad-hoc 一次性只能执行一个模块，所以 ad-hoc 一般在 ansible 中实现的是简单的任务

ansible 执行方式的另外一种就是 playbook，playbook 就像一个 shell 脚本文件，可以一次性调用多个 ansible 的模块，而且 playbook 中还可以实现条件判断、循环等高阶特性，playbook 一般都是实现复杂的任务

ad-hoc 语法格式

Markdown 复制

```
ansible 主机 -m 模块 -a '模块参数' ansible选项
eg: ansible all -m shell -a 'useradd zhangsan' -uroot -k
```

2 命令执行模块

ansible 的命令执行模块，就是相当于在被控节点直接去执行 shell 命令，所以叫做命令执行模块

而这些命令执行模块和其他的模块之间有一个很大的区别：

命令执行模块不具备幂等性

幂等性：执行一次任何或者多次执行任务，结果只要是一样的，那么就不会发生任何改变

调用 user 模块去被控节点创建用户：

第一次调用的时候，创建了用户

第二次调用的时候，已经存在的用户，结果是一样的，不会发生任何改变

- shell 模块**

简单容易上手，本质就是执行 shell 命令，也就是所谓的 linux 命令

支持一些高阶参数：如 creates、removes、chdir..

和 shell 模块一模一样，只不过有一些特殊符号不支持，比如 |、>...

执行任务的时候—>直接 SSH 远程执行命令，并没有将模块封装为 py 脚本去执行，所以 raw 模块可以在没有 python 环境下的被控节点执行任务
- raw 模块

和 shell 模块也是一模一样，但是不支持 shell 模块和 command 模块的那些参数

执行任务的时候—>直接 SSH 远程执行命令，并没有将模块封装为 py 脚本去执行，所以 raw 模块可以在没有 python 环境下的被控节点执行任务
- script 模块

在被控节点执行主控节点的 shell 脚本

实际上并不是将 shell 脚本推送到远程主机执行，而是在主控节点上读取脚本的 shell 命令，把 shell 命令推送到被控节点执行

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# cat demo.sh
#!/bin/bash
ansible all -m shell -a 'rm -rf /etc/yum.repos.d/* && \
umount /dev/sr0 && \
mount /dev/sr0 /media && \
echo [base] >> /etc/yum.repos.d/dvd.repo && \
echo name=base >> /etc/yum.repos.d/dvd.repo && \
echo baseurl=file:///media/BaseOS >> /etc/yum.repos.d/dvd.repo && \
echo gpgcheck=0 >> /etc/yum.repos.d/dvd.repo && \
echo enabled=1 >> /etc/yum.repos.d/dvd.repo && \
echo [app] >> /etc/yum.repos.d/dvd.repo && \
echo name=app>> /etc/yum.repos.d/dvd.repo && \
```

```
echo baseUrl=file:///media/AppStream >> /etc/yum.repos.d/dvd.repo && \
echo gpgcheck=0 >> /etc/yum.repos.d/dvd.repo && \
echo enabled=1 >> /etc/yum.repos.d/dvd.repo && \
yum install -y httpd && \
systemctl stop firewalld && \
setenforce 0 && \
systemctl start httpd && \
echo web > /var/www/html/index.html
,
```

3 常用模块

3.1 文件管理模块

- 1. copy
- 2. fetch
- 3. file

- file：在被控节点查看文件信息、创建 | 删除 | 修改普通文件/目录文件

touch、rm、mkdir、chmod、chown

- 1. 查看 node 节点的/etc/passwd 文件信息

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible node1 -m file -a 'path=/etc/passwd state=file'
node1 | SUCCESS => {
  "ansible_facts": {
    "discovered_interpreter_python": "/usr/bin/python"
  },
  "changed": false,
  "gid": 0,
  "group": "root",
  "mode": "0644",
  "owner": "root",
  "path": "/etc/passwd",
  "secontext": "system_u:object_r:passwd_file_t:s0",
  "size": 2243,
  "state": "file",
  "uid": 0
}
```

- 2. 在 node1 和 node2 节点上创建/opt/file 文件，要求文件的拥有者是 devops，拥有组是 root，文件权限是 777

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible node1,node2 -m file -a 'path=/opt/file state=touch owner=devops group=root mode="0777"'
```

- 3. 修改 node2 节点上的/opt/file 文件信息，将文件的拥有者修改为 root

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible node1,node2 -m file -a 'path=/opt/file state=file owner=root'
```

- 4. 在所有的节点上创建/opt/dir 目录

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m file -a 'path=/opt/dir state=directory'
```

- 5. 删除所有节点上的/opt/dir 目录文件

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m file -a 'path=/opt/dir state=absent'
```

你记住，file 模块只能一个文件一个文件的删除，不支持批量删除文件

6. 在 node1 节点上创建软链接文件/opt/passwd，源文件指定/etc/passwd

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m file -a 'src=/etc/passwd dest=/opt/passwd state=link'
```

• copy：实现控制节点和被控节点文件的传输；把控制节点的文件传输给被控节点，在被控节点自身上进行文件拷贝，支持将内容直接写入到文件中

常用参数：

- backup：在覆盖文件之前先备份一份，备份的文件名以时间命名
- content：把内容写到文件中
- dest：目标文件
- src：源文件
- remote_src：如果为 yes，则 src 表示被控节点路径，如果为 false，则 src 表示主控节点的路径
- force：是否要覆盖文件

1. 将控制节点的/etc/passwd 拷贝到所有的被控节点的/tmp 目录下

Bash 复制

```
ansible all -m copy -a 'src=/etc/passwd dest=/tmp backup=true'
```

2. 将 node1 节点的/etc/passwd 文件拷贝到自己节点的/opt 目录下，修改名/opt/passwd-node1

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible node1 -m copy -a 'remote_src=yes src=/etc/passwd dest=/opt/passwd-node1'
```

3. 使用 copy 模块将 httpd index 内容写入到/var/www/html/index.html 文件

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m copy -a 'content="httpd index\n" dest=/var/www/html/index.html'
```

• fetch：将被控节点的文件拷贝到控制节点上，只能拉取普通文件，目录是无法拷贝过来的，默认情况下，拉取过来的文件在控制节点上是以被控节点主机名的名字作为目录名字来存储的

将所有的被控节点的/etc/passwd 文件保存到控制节点的/opt 目录下

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m fetch -a 'src=/etc/passwd dest=/opt '
```

直接拉取文件，不要以目录结构存储

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m fetch -a 'src=/etc/passwd dest=/opt/ flat=true'
```

注意，flat 设置为 true，然后 dest 如果是一个目录，那么目录后面必须要带/

3.2 软件管理模块

- yum_repository 模块：管理 yum 仓库配置文件（创建配置文件往里面写内容，修改配置文件的内容）不能删除 yum 仓库配置文件
- yum 模块：管理软件包的，比如安装、更新、卸载...

• yum_repository

常用参数：

- file：指定 yum 配置文件的，不需要指定.repo 后缀
- description：对应配置文件的中 name 内容
- name：对应配置文件中的[]，仓库名
- enabled：对应配置文件中的 enabled
- gpgcheck：对应配置文件中的 gpgcheck
- gpgkey：对应配置文件中的 gpgcheck
- state：present 创建仓库以及修改的含义，absent 删除仓库

在所有的被控节点创建 yum 配置文件，要求文件名称为 dvd.repo，文件中包含俩个仓库，仓库信息如下

YAML 复制

```
[base]
name=base
baseurl=file:///media/BaseOS
gpgcheck=0
enabled=1
[app]
name=app
baseurl=file:///media/AppStream
gpgcheck=0
enabled=1
```

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m yum_repository -a 'file=dvd name=base description=base baseurl=file:///media/B
aseOS gpgcheck=0 enabled=1 state=present'

[root@adsl-172-10-0-153 ansible]# ansible all -m yum_repository -a 'file=dvd name=app description=app baseurl=file:///media/App
Stream gpgcheck=0 enabled=1 state=present'
```

• yum

可以对包进行管理，也可以对包组进行管理

常见参数：

- name：指定包或者包组的名字
- state：absent 卸载、latest 更新、present 安装

安装 nginx 软件包

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m yum -a 'name=nginx state=present'
```

安装 Development Tools 软件包组

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m yum -a 'name="@Development Tools" state=present'
```

更新系统上所有的 rpm 包版本

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m yum -a 'name="*" state=latest'
```

卸载 nginx 软件包（不会同时卸载依赖包）

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m yum -a 'name=nginx state=absent'
```

3.3 服务管理模块

- service
- systemd

systemd 模块多了 daemon_reload 参数，相当于执行了 systemctl daemon-reload

一旦修改了服务的服务单元配置文件，一定要 daemon-reload 才能够读取到

其他的参数都是一样

常用参数：

- name：操作哪个服务
- state：started、stopped、restarted、reloaded
- enabled：是否实现开机自启动

1. 重启 httpd 服务，并且实现开机自启动

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m systemd -a 'name=httpd state=restarted enabled=yes'
```

3.4 用户管理模块

1. user 模块：管理用户的，相当于 useradd、userdel、usermod
2. group 模块：管理用户组的，相当于 groupadd、groupmod、groupdel

- user 模块

创建 lisi 用户，要求 uid 是 3300，gid 是 3300，附加组是 root，描述信息是 web user，登录 shell 是/bin/bash

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m user -a 'name=lisi uid=3300 groups=root comment="web user" shell=/bin/bash state=present'
```

修改 lisi 用户信息，将其登录 shell 修改为/sbin/nologin

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m user -a 'name=lisi shell=/sbin/nologin state=present'
```

删除 lisi 用户，删除用户的要删除家目录和邮箱文件，所以这里需要额外添加一个参数 remove=yes

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m user -a 'name=lisi state=absent remove=yes'
```

- group 模块

创建用户组 itgroup，gid 是 6666

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m group -a 'name=itgroup gid=6666 state=present'
```

修改组的 gid 为 7777

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m group -a 'name=itgroup gid=7777 state=present'
```

删除用户组

Bash | 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m group -a 'name=itgroup state=absent'
```

3.5 计划任务模块

- cron 模块：相当于 crontab 周期性计划任务

常用参数：

minute

hour

day

month

weekday

job：定义执行的任务命令

user：以什么用户的身份去执行任务

state：创建/删除

cron_file：指定在哪个文件定义计划任务，要么是/etc/crontab 要么是/etc/cron.d

在 node1 节点上，每天上午 9 点 15 分，以及下午的 17 点 15 分，执行 echo hello 任务

Bash | 复制

```
[root@adsl-172-10-0-153 ansible]# ansible node1 -m cron -a 'name=job1 minute="15" hour="9,17" job="echo hello" state=present'
```

删除这个计划任务

Bash | 复制

```
[root@adsl-172-10-0-153 ansible]# ansible node1 -m cron -a 'name=job1 state=absent'
```

3.6 文件下载模块

- get_url：在被控节点下载文件使用的

常用参数：

url：指定下载的文件的路径

dest：指定下载到目的的哪个位置

Bash | 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m get_url -a 'url=https://releases.ansible.com/ansible/ansible-2.9.0.tar.gz dest=/opt'
```

3.7 解压缩模块

- unarchive：可以把控制节点的压缩包解压缩到被控节点，也可以在被控节点自身上找压缩包然后解压缩到自身另外一个目录

常用参数：

src：压缩文件的源路径，默认是在控制节点上找到的，除非 remote_src=yes

dest：解压缩后放到哪个目录下

remote_src：表示找的压缩文件是否来自于被控节点本身，如果 remote_src=yes，则 src 是在被控节点本身搜索的压缩文件，如果 remote_src=no，则 src 表示控制节点的压缩文件。默认 remote_src=no

Bash 复制

```
ansible all -m unarchive -a 'src=demo.tgz dest=/opt'
ansible all -m unarchive -a 'remote_src=yes src=/opt/ansible-2.9.0.tar.gz dest=/opt'
```

3.8 文件同步模块

- synchronize 模块：相当于执行 rsync 命令
 - 将远程主机 被控节点的文件拉取到控制节点上
 - 将控制节点的文件推送到被控节点

底层其实走的是 rsync 命令，而 rsync 走的是 SSH 协议

常用参数：

mode：pull/push，默认是 push。如果是 push 的话，那么 src 表示控制节点的文件路径；如果是 pull 的话，那么 src 表示被控节点的路径

src：文件源路径

dest：文件目标路径

传输目录文件：

将控制节点的/etc/passwd 文件同步到所有的被控节点的/opt 目录下

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m synchronize -a 'src=/etc/passwd dest=/opt'
```

将所有的被控节点的/opt/passwd 拉取到控制节点的/tmp 目录下

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m synchronize -a 'mode=pull src=/opt/passwd dest=/tmp'
```

```
[root@adsl-172-10-0-153 ansible]# ansible all -m synchronize -a 'src=/etc/passwd dest=/opt'
```

传输目录文件：

传输的如果是目录，那么对于 src 来说，后面路径带不带/是存在不同的意义的

如果带上了/，表示同步这个目录下所有的文件

如果没带/，表示同步这一整个目录

将控制节点的/opt/目录同步到被控节点的/opt 目录下

Bash 复制

```
[root@adsl-172-10-0-153 ansible]# ansible all -m synchronize -a 'src=/opt dest=/opt'
这个没带/，同时的是一整个opt目录
[root@adsl-172-10-0-153 ansible]# ansible all -m synchronize -a 'src=/opt/ dest=/opt'
这个带了/，同步的是/opt目录下的所有文件
```

